Author: Micheal Lanham

SECTION 1

Architecting Multi-Agent Systems

- **Three canonical shapes:** Production multi-agent design centers on agent flow (assembly line), orchestration (hub-and-spoke), and collaboration (peer teams), each committing differently to authority and context sharing.
- **Decision-making:** Decision-making is the power to commit to the next action; architecture must specify which agent holds that authority and what context it sees when deciding.
- **Control:** Control is execution authority—planners may decide without calling tools, while workers may execute without choosing the next step.
- **Communication:** Communication governs what flows between agents; open shared context aids coordination but causes context dilution and token fan-out across calls.
- **Coordination strategies:** Beyond linear flow, systems combine sequential pipelines, parallel delegation, hierarchical coordination, iterative critique, debate, voting/best-of-N, role-play, conditional routing, and peer networks as situational patterns.
- **Agency versus constraint:** Effective design starts with minimal agency and tightens boundaries on irreversible actions, expanding autonomy only where reliability is demonstrated.

SECTION 2

Balancing Agents with Agentic Flows

- **Splitting overload:** A single agent overloaded with tools and instructions is often refactored into a specialized agent flow for clarity, accuracy, and lower per-step token cost.
- **Transformation triggers:** Common drivers include tool overload, role specialization, and cost/latency reduction from scoped tools and leaner prompts.
- **Prompt chaining with tools:** Agent flows resemble prompt chaining, but each stage can reason and invoke tools before passing concise outputs downstream.
- **Deterministic decision points:** Critical branches—such as whether search returned sources—should be implemented in typed code checks, not left to stochastic LLM judgment.
- **Structured I/O:** Pydantic output types and labeled dictionaries make pass/fail logic reliable and reduce the need to parse free-form prose between stages.

SECTION 3

Understanding Handoffs in Agent Flows

- **Communication patterns:** Flows may use a shared conversation thread, explicit pass-off routing in orchestration code, or SDK handoffs that transfer control inside the runtime.
- **Handoff mechanics:** The OpenAI Agents SDK `handoffs` field plus instructional prompts lets agents chain internally without manual `Runner.run` per stage.
- **Observability:** `draw_graph` and dashboard Traces expose agent graphs, tool chains, and latency hotspots for debugging complex flows.
- **Handoff monitoring:** `on_handoff` callbacks log or process transferred payloads at boundaries; callbacks observe, while guardrails block or correct unsafe data.
- **Naming discipline:** Stable agent naming conventions and eventually an agent registry become essential as flow complexity grows past what a team can hold in memory.

SECTION 4

Validating Agent Flows with Guardrails

- **Semantic circuit breakers:** Guardrails halt risky execution before bad data or actions propagate, distinct from post-hoc output filtering alone.
- **Layered validation:** Production posture combines regex/schema checks, small classifier models, and LLM guardrails only where validation requires understanding.
- **Input and output guardrails:** SDK `input_guardrails` and `output_guardrails` return tripwires that raise exceptions agents or orchestration code can catch and recover from.
- **Agent-as-guardrail:** A dedicated guardrail agent can enforce natural-language policies on inputs or outputs, trading flexibility for nondeterminism that still needs schemas and tests.
- **Pass-off and handoff risks:** Boundaries must guard against shape failures, context leakage, and instruction injection from upstream tool or agent content on high-stakes transfers.
- **Retry with limits:** Bounded retry loops after guardrail trips can improve plans but must cap iterations and use backoff for transient infrastructure failures.